SECURE

SOFTWARE SYSTEM

(MITS 5400G/CSCI 6320G)

Submitted to,

Name: Dr. Pooria Madani

Faculty of Business and Information Tech

Submitted by,

Name: Avishkar Sharma

Assignment 3

Table of Contents

Q.NO.1

Q.NO.1(a) DAC Scheme

Traditional DAC is very user-centric and one where permission control is based on the user. It simply refers to a big access matrix in which every single subject is mapped to every object at each intersection, with access rights defined on an individual basis.

The following entities are the variables found in a traditional DAC scheme.

N = Number of job positions

U_i = Number of individual users

P_i = Number of permissions required

Permission management is done based on users; every position holder must be closely associated with each permission appropriate for their position. The count of associations for every position is the result of the multiplication of the number of users and permissions required for that position are:

$$\text{For N number of users} = \sum_{i=1}^{N} (U_i)$$

$$\text{For N number of permissions} = \sum_{i=1}^{N} (P_i)$$

The total count of relationships throughout all N positions is as follows:

$$\text{DAC calculation for N job is} = \sum_{i=1}^{N} (U_i \times P_i)$$

For example,

Let's take Users (U_i) is 5 and Permissions (P_i) is 3 then, for DAC each individual user to every specific permission so the matrix must be 5 by 3 i.e.

User	Permission 1	Permission 2	Permission 3
User 1	Yes	Yes	Yes
User 2	Yes	Yes	Yes
User 3	Yes	Yes	Yes
User 4	Yes	Yes	Yes
User 5	Yes	Yes	Yes

Total entries to manage are 15.

Q.NO.1(b) RBAC Scheme

RBAC (Role based Access Control) detaches access from the user's identity. Instead of applying rights to each user, rights are assigned to roles which represent the job functions and then users are assigned to those roles. So, in RBAC scheme, the complexity is reduced by using roles as an intermediary, which is the sum of permission to role and user to role.

In RBAC scheme, the variables given are:

N = Number of job positions

U_i = Number of individual users

P_i = Number of permissions required

1. Permission to Role: Each role is assigned to its specific permissions. Since there are N jobs positions, and each position I requires P_i permissions, there are:

$$\sum_{i=1}^{N}(P_i)$$

2. User to Role: Each user is assigned to their respective job position. Since there are U_i users in each position, there are:

$$\sum_{i=1}^{N}(U_i)$$

Hence the total number of relationships in an RBAC system is:

$$\text{RBAC calculation for N job} = \sum_{i=1}^{N}(P_i) + \sum_{i=1}^{N}(U_i)$$

For example,
In RBAC, we introduce the Role as an abstraction layer. We split the matrix into two smaller parts. Let's take Users (U_i) is 5 and Permissions (P_i) is 3 then,

Permission to Role: Here we define the permission for the job position(Role) just once.

Role	Permission1	Permission 2	Permission 3
Role A	Yes	Yes	Yes

User to Role: Here we define each user to the Role.

User	Role A
User 1	Yes
User 2	Yes
User 3	Yes
User 4	Yes
User 5	Yes

Q.NO.2

The primary differences between Discretionary Access Control from Mandatory Access Control lie in the policy source. DAC allows object owners to delegate access whereas MAC mandates access based on system-wide security label, regardless of owner preference.

DAC (Discretionary Access Control):

- Control: Access to an object is at the discretion of its owner. For instance, an owner of a file can control who is able to read or make changes to this file and hand these permissions down to others according to their decision-making.
- Flexibility: Being user-friendly and flexible, DAC is best suitable for deployment in a small-scale environment, such as personal computing systems or small workgroups. This is so because the flexibility of DAC to allow read or write access to any resource might not be rigorously applied at a larger level that would certainly require centralized control.

MAC (Mandatory Access Control):

- Control: The access will also be controlled by the system itself, following centrally defined security policies. Users will not have the option of bypassing these policies for their own data. That's what makes it "mandatory": the final decision about access is with the system and not the user.
- Stricter Enforcement: MAC basically depends on the security labels that are appended to subjects and objects, such as Top Secret, Confidential, and Unclassified. When a request for access is initiated by a subject, the system must check their clearance against the sensitivity label of the object and automatically allow or deny access depending on whether meeting the defined clearance or not.
- Use Case: Implementation of MAC is in military or highly sensitive government systems with stringent security and where it must be mandatory and enforced by OS means and not by the user.

Comparison of Mechanisms

Feature	**Discretionary Access Control(DAC)**	**Mandatory Access Control(MAC)**
Basis for Decision	Based on Identity and authorization	Based on security labels and security clearance of resources
Enforcement Method	Implemented using an access matrix mapping	Enforced by the system through comparison of sensitivity levels and eligibility
Nature of Rights	Flexible and owners can pass rights to others at their discretion	Fixed, entities cannot transfer their own access rights to others.

Table 1: Comparison between DAC and MAC

Q.NO.3

Salt offers benefit of preventing pre-computation attacks and reduces duplicate password reuse. In this process, each individual character strings across have unique username and password combination, giving a perception that a new hash table is computed for each user, which effectively makes precomputed Rainbow tables useless.

1. Prevents Bulk Cracking: Even if the attacker knows what the salt will be, that doesn't really help much in cracking the whole database. The attacker, in this case, would simply need to re-calculate the hashes for all possible values one way to put that is that this effectively makes the attack 4096 times slower than usual.
2. Defeating Pre-computation: In the absence of salting, an attacker can precompute hashes for a dictionary of common passwords, technique known as creating a Rainbow Table. Because the hash for 'Password123' will always produce the exact same hash, now a single hash can effectively crack millions of accounts simultaneously.
3. Brute-Force Prevention per Instance: The salt affects the final hash output, denying attackers the use of a single precomputed Rainbow Table for all the passwords. Each new table needs to be created with each user account, taking a longer time to do large-scale attacks.
4. Unique Hashes: The unique hashes used guarantee that even for the same passwords used by two users, the result is a completely different hash. This effectively masks all relationships between accounts. Therefore, even successfully cracking one password will not yield any information about the possibility of other users having an equivalently weak password.

In conclusion, while the salt is stored in plain text and known to both user and attacker, an attacker will not be able to exploit this knowledge. Rather, it will probably have the attacker spending massive computational time on each account, which in all other aspects makes bulk-cracking attacks such as the Rainbow Tables worthless. Indeed, it does so by making an attack on the database so much more expensive and time-consuming.

Q.NO.4

Q.NO.4(a) Brute Force Search Space calculation

The formula for finding the size of the search space for a brute-force attack is: : R^L, where R is the count of possibilities and L is the Key length.

It is given that the passwords are alphanumerical meaning there are, Uppercase Letters (A-Z), Digits (0–9) no lower cases are considered because the passwords are converted into all uppercase and a null symbol. Thus, Total possible character per position (R) = 26 +10+1 = 37.

The problem stated that the Key length, L, is in 14-character password but it is broken into two halves of 7-character strings each. Thus, for every string it becomes independent hence L=7.

Search space for first 7-character strings = 37^7 which is approximately $9.49*10^{10}$

Search space for last 7-character strings = 37^7 which is approximately $9.49*10^{10}$

Total search space=$2*37^7 = 1.898*10^{11}$

Note: The total space is $37^7*37^7 = 37^{14}$, but according to step 4 which mentions that password is divided into two halves so exploit complexity is $2*37^7$

Q.NO.4(b) Major Drawbacks of the LM Hash scheme

This scheme may be described as having catastrophic cryptographic and design flaws, even though the theoretical search space of the LM hash scheme is great. Some of the drawbacks are as follows:

- No Salt: Lack of salt catastrophically damages the LM hash. So, when the encryption target is the string "KGS!@#$%," the hash produced will be the same for every instance. This means that an attacker could use precomputed Rainbow Tables to simultaneously break into corresponding stolen databases at the same time, thus dissolving the large number of breaches.
- Passwords are case insensitive: Because the scheme first converts every password to uppercase before hashing which reduces the effective character set to 37 (uppercase, digits and a null) instead of 63 (lowercase, uppercase, digits and a null). This drastically reduces search space. For instance, if a user uses a password of “PassWord1” is having the hash of “PASSWORD1” stored. This directly relates to the point of reducing the character set size.
- Password Splitting: This is the most serious flaw where splitting the 14-character password into two 7-character halves and concatenating them. The scheme effectively produces two independent hashes. The attack really doesn't have to crack a 14-character password only two 7-character ones, essentially making password with 7-character segments).
- Cryptographic Primitive (DES): The scheme makes use of a cryptographic hash function DES that isn't meant for such use; moreover, DES is outdated today, and it can be easily cracked using brute-force techniques on just a 56-bit key space.
- Null-padding Signature: Step 3 in the process pads passwords shorter than 14 characters with nulls. Then the hash of the second half would always be either 7 characters or less under DES ("KGS!@#$%", Key_from_Nulls). This constant leak the length of the password down at the attacker.

- No work Factor: Modern password hashing algorithms like bcrypt, scrypt, Argon2 were built with an intent to be slow and have a work factor that can be ratcheted up with improved hardware. LM doesn't have those tricks up its sleeve, just simply fast, which benefits the attackers directly.

In essence, the LM hash violates practically every rule of modern password storage. Its design of case insensitivity, password splitting, no salt, weak crypto all decreases effective search space and cost attacks from the theoretical maximum of $9*10^{21}$ to $1.898*10^{11}$operations, making it completely insecure for a modern environment.

Q.NO.5

Q.NO.5(a) Access Policies (ABAC)

Access policies on the ABAC rules are decided based on three different types of attributes:

- User attributes: For a user, attributes include Age which could be calculated from Date of Birth and Membership Type which could be either Regular or Premium.
- Object Attributes: It includes the attributes of Movies Rating : G, PG-13, R and Release Date.
- Environmental Attribute: Current date (For calculating the age of the user and movie).

To implement role-based access control, these role configurations must be engineered so that every possible combination of the above attributes is covered. One such predefined user type with a specific movie accessing permission will be captured in each role.

Access Rule: Two rules have been framed Rule R1 and Rule R2, where:

Rule R1: Here, access is controlled according to age and movie rating.

- Users above the age of 17 years get to watch all movies.
- Users between the age of 13 and 16 (both inclusive) can view PG-13 and G movies.
- Users under the age of 13 can only access movies rated G.

Rule R2 states that access is decided based on the membership type and the release date of the movie.

- Premium users have access to all movies, regardless of release date.
- Regular users can watch only those movies that have been released for more than 30 days.

Rule R3 requires both R1 and R2 to be satisfied.

Defining Age Based Role

Role Name	Age Group	Description
Child	<13	Can only access G-rated movies
Teen	13-16	Can access G and PG-13 movies
Adult	≥17	Can access G and PG-13 movies

Table 5.1: Defining the Age Group

Defining Subscription Based Role

Subscription	Allowed Releases	Description
Regular	Old (>30)	Can only access movies older than 30 days
Premium	All (New and Old)	Can access both old and new releases.

Table 5.2: Defining Subscription Group

Calculating each user's age, based on their Date of Birth. Present date:10 March 2026

User	Date of Birth	Age	Age Group	Subscription
alan_j123	1949-01-02	77 years	Adult (≥17)	Regular
sarah_t33	1966-05-28	59 years	Adult (≥17)	Premium
jason_jj	2018-02-02	8 years	Child (<13)	Premium
brienna	2006-02-15	20 years	Adult (≥17)	Regular

Table 5.3:User's Age, Age Group and Subscription Attributes

User to Role Assignment (Aged based only)

User	Child	Teen	Adult
alan_j123			Yes
sarah_t33			Yes
jason_jj	Yes		
brienna			Yes

Table 5.4:Role name and age group

Now, we determine Movie attributes where we calculate the attribute of each movie.

Present Date: 10 March 2026

Movie Title	Rating	Release Date	Status
Ratatouille	G	March 10, 2020	Old (>30)
Wall-E	G	March 7, 2026	New (≤30)
1917	R	March 10, 2022	Old (>30)
The Goonies	PG-13	March 10, 2021	Old (>30)
Hidden Figures	PG-13	March 7, 2026	New (≤30)
A Man Called Otto (R, New)	R	March 7, 2026	New (≤30)

Table 5.5: Movie Rating, Release Date and Status

Age Role to Role Assignment (Movie Based only)

Role	G Movies	PG-13 Movies	R movies
Child	Yes		
Teen	Yes	Yes	
Adult	Yes	Yes	Yes

Table 5.6: Age to Role Assignment (Movie)

Final Access Matrix

User	Age Role	Subscription	Ratatouille (G, Old)	Wall-E (G, New)	1917 (R, Old)	The Goonies (PG-13, Old)	Hidden Figures (PG-13, New)	A Man Called Otto (R, New)
Alan_j123	Adult	Regular	Yes	No	Yes	Yes	No	No
Sarah_t33	Adult	Premium	Yes	Yes	Yes	Yes	Yes	Yes
Jason_jj	Child	Premium	Yes	Yes	No	No	No	No
Brienna	Adult	Regular	Yes	No	Yes	Yes	No	No

Table 5.7: Final Access Matrix

Age Role Permission:

Child – Only G movies

Teen – G and PG-13 movies

Adult – G, PG-13 and R movies

Subscription Constraint

Premium users – Can access both new and old releases

Regular users – Can only access old releases (>30 days)

Q.NO.5(b)Possible scenarios for Access Policies (ABAC)

Real-world systems' access control matrices are not static, so they need to evolve with changes in users, media, and membership type. The following scenarios can take place:

- Scenario 1: New User Registration, a new user will be assigned to the relevant composite role, considering their age and membership. This addition would reflect in the new row of the User-to-Role Assignment Matrix, so that the user could access the content of the system.
- Scenario 2: User's Age (Birthday), a user surpasses one age category and grows into another, for instance, from a Child to above 13 years of age. This should automatically change their entry in the User-to-Role Assignment Matrix, enabling access to new content ratings corresponding to eligibility.
- Scenario 3: Change in Membership Type, if a user upgrades or downgrades a subscription, it should affect their role assignment in the User-to-Role Assignment Matrix, so that their current access to respective content is updated instantaneously.
- Scenario 4: New movie added to the Catalog, in case a movie is added, we must first find which roles have all the required attributes for the entitlement to its viewing. This will require adding a new column to the Role-to-Permission Access Matrix.
- Scenario 5: Change in Movie Status (New to Old), once a movie becomes older than thirty days, it becomes open to all and is no longer a restricted movie for Premium members. So, it becomes necessary to update the Role-to-Permission Access Matrix in such a way that effective access to that movie column is given for the relevant "Regular" roles.
- Scenario 6: User Account Deletion, each deletion of an account should eliminate the corresponding record relating to the user within the user-to-role assignment matrix row to avoid unauthorized entries.

Q.NO.6

SSH is often targeted by attackers intending to gain unauthorized access through brute-force attacks. This includes both automated methods of guessing usernames and passwords. A careful analysis should be done on the SSH logs so that suspicious activities can be identified for proper counteracting measures. This assignment has given us a dataset: OpenSSH_2k.log_structured.csv with around 2000 data, to discover potential threat actors and temporal patterns of failed authentication attempts. The analysis will be performed on Python in Kali Linux, using pandas and matplotlib to manipulate the data and regular expressions to match.

Q.NO.6(a) Threat Actor Identification

- Brute Force attacks are an example where the attacker fills the system with multiple common usernames to gain access like “admin”, “root”, “user”.
- An entry in the log file is made after every failed attempt with an invalid username, containing “Invalid User” the attempted username and the source IP address of user trying SSH.
- By analyzing the provided CSV file, we can identify
    - Automated attacks: High number of attempts from single IP address
    - Distributed attacks: Patterns across multiple source addresses
    - Attack Persistence: Which IPs were attempting to access frequently

I have analyzed the provided OpenSSH_2k.log_structured CSV file that contains login attempts records to identify possible sources of brute force attack. To filter the invalid usernames and having failed login attempts, the focus of this assignment was on detecting and aggregating such attempts per IP address. The emphasis here is to identify which of them have exceeded the threshold of 10 failures. The security analysis is focused on identifying IP addresses that show unusual activity of potentially being threat actors trying to gain unauthorized access to a system.

The python command used is shown below and I have attached the exact python code on the appendix section of this document.

```
┌──(root㉿kali)-[/home/avishkar/Assignment_3]
└─# cat assignment_3.a.py
import pandas as pd

# Load the provided CSV file for analysis
df = pd.read_csv('OpenSSH_2k.log_structured.csv')

# Using regular expression to locate "Invalid user" authentication failures and capture the source IP address from each log
regex_pattern = r'Invalid user \S+ from (\d+\.\d+\.\d+\.\d+)'
df['Extracted_IP——Hit_Count'] = df['Content'].str.extract(regex_pattern)

# Applying filter and returning the count of matching entries
failed_attempts = df.dropna(subset=['Extracted_IP——Hit_Count'])
ip_counts = failed_attempts['Extracted_IP——Hit_Count'].value_counts()

# Keeping only records where the count met or exceeded the threshold of 10
threat_actors = ip_counts[ip_counts ≥ 10]

#Printing the Threat_Actors
print("Part A output: Attacker's IP and Hit Count")
print(threat_actors)
```

Figure 6.1:Python Code to identify failed logins with invalid usernames and respective IP addresses.

Explanation of Python code used:

Step 1: Firstly, load the CSV file for analysis.

The read_csv function loads the CSV file into a Pandas DataFrame. By this function, structured data CSV is read thus making it accessible for analysis with all its parts for each log entry, like Date, Time and content arranged in separate columns for easy treatment.

Step 2: Using Regular expression to Identify the invalid user and capture source IP

A regex pattern is defined to match the log entries for Invalid user authentication failures. In this pattern, invalid user is the literal text by which a match will start at the beginning of the log entry. \S+ is used to capture the username being attempted. 'from' is the literal text that matches what precedes the IP address; then \d+ is used to capture a source IPv4 address with escaped dots being periods in the IP address.

Step 3: Applying a filter and returning the count of matching entries

Here, regex pattern is used on the 'Content' column by applying the string.extract method from pandas to create a new column with the extracted IP in the matching entries and NaN values in those that did not match. Dropna() is used as a filter for nonmatching rows to get a dataframe that contains only the "Invalid user" authentication failure entries. This uses the value_counts() method to calculate the number of attempts per IP on the extract's columns from the unique IP addresses and returns a Series with unique IP addresses and their respective attempt count.

Step 4: Working with IP addresses that only have over 10 records

For the output part, the specified threshold in the assignment is enforced through Boolean indexing to filter and store the values with only those IP addresses where 10 or more invalid attempts have been made. This helps to identify the most persistent known bad actors in the dataset who need immediate action like blocking at the firewall level or adding to threat feeds.

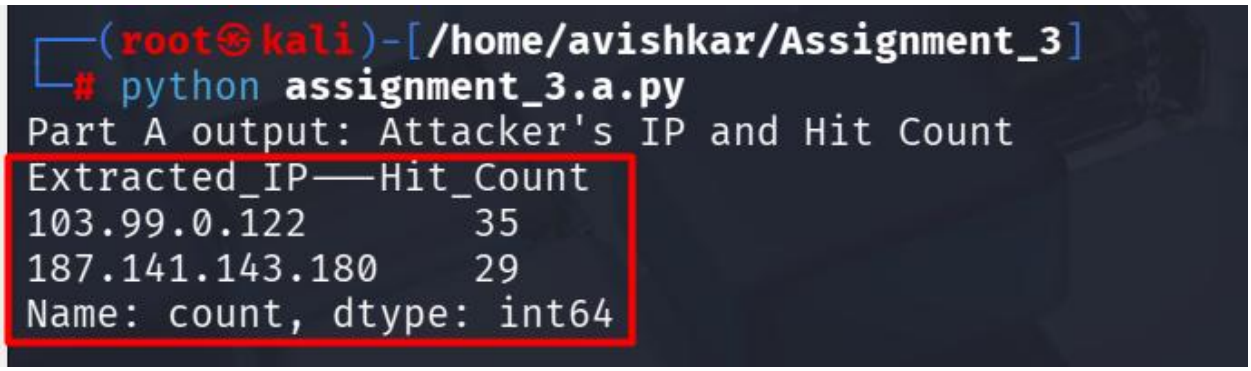

Figure 6.2:Output showing Attacker's IP and Hit count

This analysis is performed post filtering of the dataset by EventID 13 using regex to allow only unique entry for each distinct login effort. This excluding secondary system logs, such as E10 or E12, would prevent any possibility of adding extra push to the number counted. This ensures an accurate account of the real threats posed instead of allowing for multiple redundant system processing notifications.

Q.NO.6(b) Investigating the timing Column

To further investigate the administrator's suspicion of timing pattern in the failed authentication attempts from Part A, we first analyze the Time attribute in the dataset. In python, with the help of Pandas and Matplotlib libraries, hourly distribution of invalid username failures was calculated. The python command used is shown below and I have attached the exact python code on the appendix section of this document.

```
┌──(root㉿kali)-[/home/avishkar/Assignment_3]
└─# cat assignment_3.b.py
import pandas as pd
import matplotlib.pyplot as plt

# Loading and applying the  Filter
df = pd.read_csv('OpenSSH_2k.log_structured.csv')
failed = df[df['EventId'] == 'E13'].copy()
failed['Hour'] = pd.to_datetime(failed['Time'], format='%H:%M:%S').dt.hour

# Filtering to time from  06:00 to 12:00
filtered = failed[(failed['Hour'] ⩾ 6) & (failed['Hour'] ⩽ 12)]
counts = filtered.groupby('Hour').size().reindex(range(6, 12), fill_value=0)

# Console output to see the number of hitcounts
print(f"\n{'Hour':<10} | {'Attempts':<10}")
print("-" * 22)
for hour, count in counts.items():
    print(f"{hour:02d}:00-59 | {count:<10}")
print("-" * 22)
print(f"{'TOTAL':<10} | {counts.sum():<10}\n")

# Plotting the graph
plt.figure(figsize=(9, 5))
bars = plt.bar(counts.index, counts.values, color='#E74C3C', alpha=0.8)
plt.title('Login Attempts (06:00 - 12:00)', fontsize=14, pad=15)
plt.xlabel('Time (24h)')
plt.ylabel('Count')
plt.grid(axis='y', linestyle='--', alpha=0.3)
plt.xticks(range(6, 13), [f"{i}:00" for i in range(6, 13)])
plt.tight_layout()
plt.show()
```

Figure 6.3:Analyzing the time column

Explanation of the Code:

Step 1:Load and initialize data

The CSV file is loaded into a Pandas Dataframe using read_csv(). For data visualization, we need to import matplotlib.pyplot. This will be initial setup for parsing the structured log data.

Step 2: Isolate Unique Security Events

In this step, dataset was filtered to EvenId contains E13 for data integrity, as this means 'Invalid user'. This approach identifies singular authentication initiation events. This way, by focusing on just the primary eventide rather than looking for keywords matched, it would eliminate redundant logs, and the count would be only representation of the true frequency of a unique attack.

Step 3: Normalizing Temporal Data

Time columns were converted into datetime objects using pd.to_datetime(). Hour information to group attempts per 24 hours in a day: dt.hour. The reindex() function is used to filter for the requested window '06:00-12:00' and re-index to fill in all the missing hours with zero attempts.

Step 4: Console output to see the hit counts:

This part separates the unauthorized login attempts by filtering for Event Id 13,. The timestamps are parsed to extract the hour and serially breakdown of hours. The figure below shows the output generated:

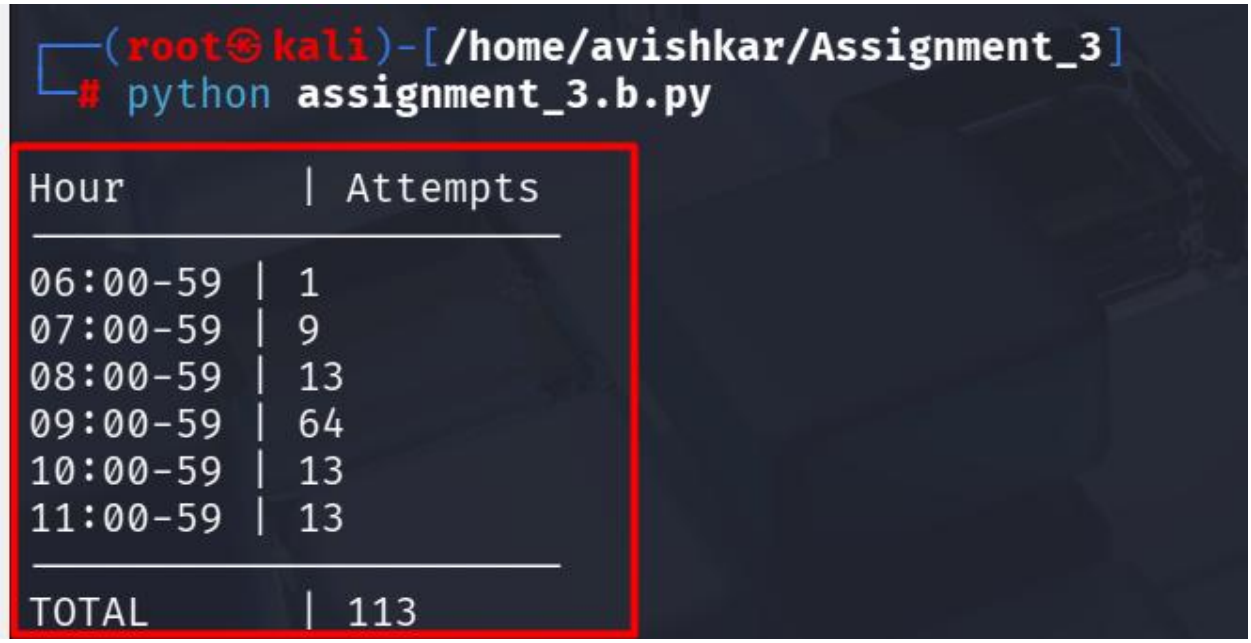

Figure 6.4: Output of the console

Step 5: Plotting Hourly Distribution

Subsequently a bar chart was generated to have a visual representation of how attacks were occurring with frequency. It should be labeled on both axes; the time format is in 24 hours each bar is showing to the number of hit counts. The graph below shows the output of the graph:

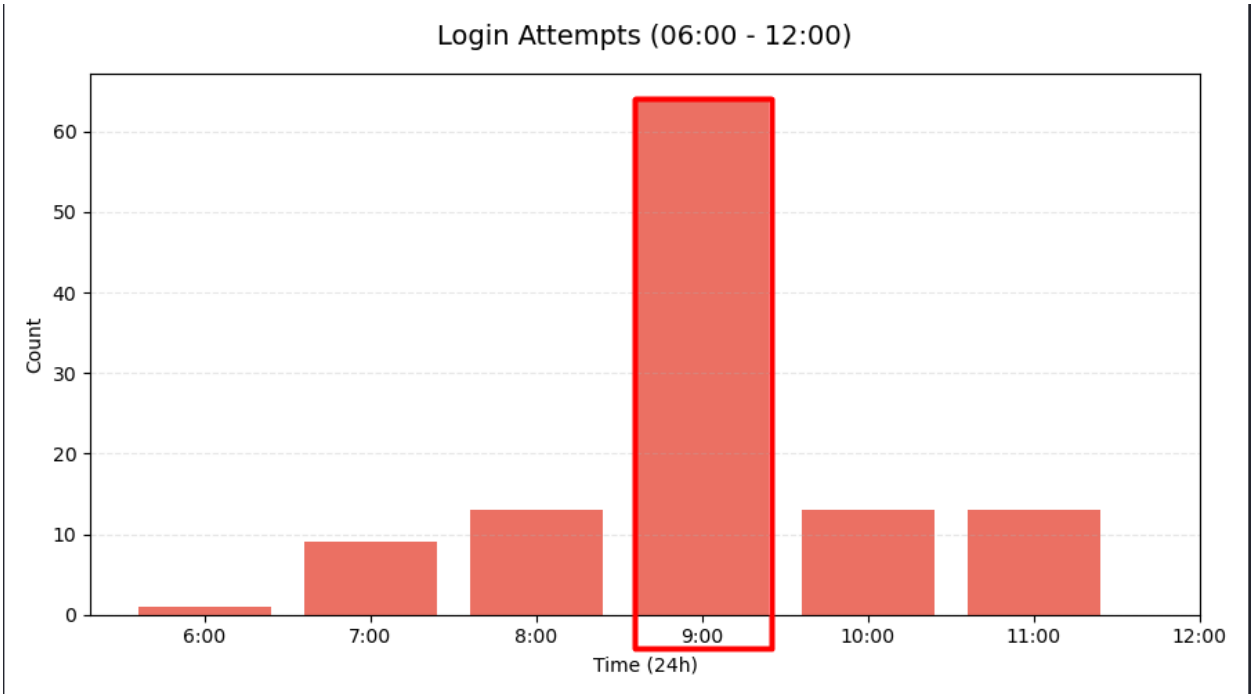

Figure 6.5: Plotting hourly distribution

Peak Activity

The attacks were most heavy between 6:00 AM to 12:00 PM, peaking at 9:00 (64 attempts). Evidently, from there, the directionality goes down quite sharply, with next to no attempts post-12:00. Attack concentration in the morning might catch peak administrative weakness hours; therefore, proactive defense should employ rate limiting or enhanced monitoring during the high windows of potential risk.

These attacks during the morning hours provide valuable insights for defense planning team. With this kind of information in when an attack is most liable to occur, a company can avail itself of an opportunity to adjust resources proactively, firewall rules, and additional verification during high-risk periods. This is the way that the proactive approach reconceptualizes raw log data as actionable security intelligence, an improvement against brute-force attacks.

Appendix

Python Code for Part 6(a)

```
import pandas as pd

# Load the provided CSV file for analysis

df = pd.read_csv('OpenSSH_2k.log_structured.csv')

# Using regular expressions to locate "Invalid user" authentication failures and capture the source IP address from each log entry

regex_pattern = r'Invalid user \S+ from (\d+\.\d+\.\d+\.\d+)'

df['Extracted_IP---Hit_Count'] = df['Content'].str.extract(regex_pattern)

# Applying filter and returning the count of matching entries

failed_attempts = df.dropna(subset=['Extracted_IP---Hit_Count'])

ip_counts = failed_attempts['Extracted_IP---Hit_Count'].value_counts()

# Keeping only records where the count met or exceeded the threshold of 10

threat_actors = ip_counts[ip_counts >= 10]

#Printing the Threat_Actors

print("Part A output: Attacker's IP and Hit Count")

print(threat_actors)
```

Python Code for Part 6(b)

```
import pandas as pd
import matplotlib.pyplot as plt
# Loading and applying the  Filter
df = pd.read_csv('OpenSSH_2k.log_structured.csv')
failed = df[df['EventId'] == 'E13'].copy()
failed['Hour'] = pd.to_datetime(failed['Time'], format='%H:%M:%S').dt.hour
# Filtering to time from  06:00 to 12:00
filtered = failed[(failed['Hour'] >= 6) & (failed['Hour'] <= 12)]
counts = filtered.groupby('Hour').size().reindex(range(6, 12), fill_value=0)
# Console output to see the number of hitcounts
print(f"\n{'Hour':<10} | {'Attempts':<10}")
print("-" * 22)
for hour, count in counts.items():
    print(f"{hour:02d}:00-59 | {count:<10}")
print("-" * 22)
print(f"{'TOTAL':<10} | {counts.sum():<10}\n")
# Plotting the graph
plt.figure(figsize=(9, 5))
bars = plt.bar(counts.index, counts.values, color='#E74C3C', alpha=0.8)
plt.title('Login Attempts (06:00 - 12:00)', fontsize=14, pad=15)
plt.xlabel('Time (24h)')
plt.ylabel('Count')
plt.grid(axis='y', linestyle='--', alpha=0.3)
plt.xticks(range(6, 13), [f"{i}:00" for i in range(6, 13)])
plt.tight_layout()
plt.show()
```